

1. [2p] Dado el siguiente código Java,

- a) Dibujar un diagrama de clases que muestre todas las clases/interfaces involucradas (salvo Object) y sus relaciones.
b) ¿Hay comportamiento polimórfico en el código? Indicar dónde y explicar brevemente.

LectorCSV.java

```
1 import java.io.BufferedReader;
2 import java.io.IOException;
3 import java.util.List;
4 import java.util.Arrays;
5
6 public class LectorCSV {
7     private BufferedReader entrada;
8
9     public LectorCSV(BufferedReader entrada) {
10         this.entrada = entrada;
11     }
12
13     public List<String> lerFila() throws IOException {
14         String linea = entrada.readLine();
15         if (linea == null) {
16             return null;
17         }
18         String[] valores = linea.split(",");
19         return Arrays.asList(valores);
20     }
21 }
```

Main.java

```
1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.IOException;
4 import java.util.List;
5
6 public class Main {
7     public static void main(String[] args) throws
8         IOException {
9         BufferedReader br = new BufferedReader(new
10             FileReader("datos.csv"));
11         LectorCSV lector = new LectorCSV(br);
12
13         List<String> fila;
14         while ((fila = lector.lerFila()) != null) {
15             System.out.println(fila);
16         }
17         br.close();
18     }
19 }
```

Nota: las definiciones de las clases e interfaces usadas de la biblioteca estándar son las siguientes:

```
public interface List<E> { ... }
public class BufferedReader extends Reader {
    public BufferedReader(Reader in) { ... }
    String readLine() { ... }
    ...
}
```

```
public class FileReader extends InputStreamReader { ... }
public class InputStreamReader extends Reader { ... }
public abstract class Reader { ... }
```

2. [1p] Considerar el siguiente código. Identificar un principio de diseño que se viola, indicar cuál es y justificar. Proponer una solución para evitar la violación del principio (no es necesario escribir el código).

```
1 public class Factura {
2     private String empresa;
3     private List<Item> items;
4
5     public Factura(String empresa, List<Item> items) {
6         this.empresa = empresa;
7         this.items = items;
8     }
9
10    public void agregarItem(Item item) {
11        items.add(item);
12    }
13 }
```

```
13 public void exportar(String ruta, String formato) {
14     OutputStream archivo = new FileOutputStream(ruta);
15     if (formato.equals("PDF")) {
16         exportarPDF(archivo);
17     } else if (formato.equals("HTML")) {
18         exportarHTML(archivo);
19     } else if (formato.equals("TXT")) {
20         exportarTXT(archivo);
21     }
22 }
23
24 private void exportarPDF(OutputStream archivo)
25 { ... }
26 private void exportarHTML(OutputStream archivo)
27 { ... }
28 private void exportarTXT(OutputStream archivo)
29 { ... }
30 }
```

3. [2p] Dada la siguiente base de datos Prolog, dibujar el árbol de búsqueda e indicar el resultado de la consulta:
?- lo_quiero(X).

```
1 es_nuevo(iphone_13).
2 es_nuevo(galaxy_s21).
3 es_potente(iphone_13).
4 es_potente(pixel_6).
5 es_barato(galaxy_s21).
6 es_barato(pixel_6).
7
8 lo_quiero(X) :- es_nuevo(X), es_potente(X), es_barato(X).
```


4. [2p] Dado el siguiente código Clojure,

- ¿Cuál o cuáles funciones (con nombre o anónimas) son puras/impuras? Justificar.
- ¿Cuál o cuáles funciones (con nombre o anónimas) son de orden superior? Justificar.
- ¿La salida del programa es determinística? Justificar.

```
1 (defn con-registro [f]
2   (let [log (atom [])]
3     [log
4      (fn [x]
5        (swap! log conj x)
6        (f x))])])
7
```

```
8 (let [[log cuad] (con-registro
9               (fn [x]
10                 (* x x)))
11      a (future (cuad 10))
12      b (future (cuad 20))
13      resultado (+ @a @b)]
14   (println "El resultado es:" resultado)
15   (println "La función fue invocada" (count @log)
16           "veces.")
16   (println "Log:" @log))
```

5. [2p] Dada la siguiente expresión lambda,

- Identificar las variables libres y ligadas.
- Mostrar los pasos de reducción usando orden normal (β -redex más externa desde la izquierda) hasta llegar a la forma normal (si existe). Aplicar reducciones α según sea necesario para evitar conflictos de nombres.

$(\lambda x f.f x) (\lambda q.f) (\lambda a.a a)$

6. [1p] Dado el siguiente código JavaFX:

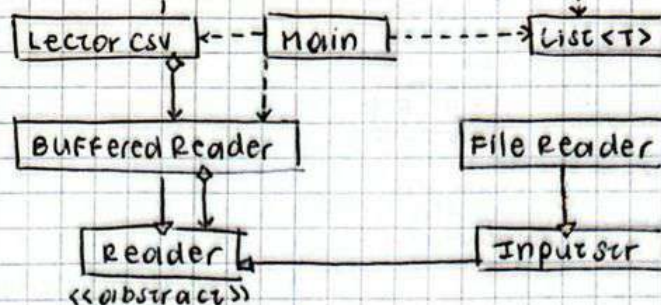
- Dibujar el árbol correspondiente al Scene Graph.
- Explicar brevemente el funcionamiento del programa. ¿Qué sucede cuando el usuario interactúa con la interfaz? ¿Cómo se logra este comportamiento?

```
1 public class App extends Application {
2   @Override
3   public void start(Stage stage) throws Exception {
4     Label etiqueta = new Label("Ingrese un color en formato #RRGGBB:");
5     TextField color = new TextField("#ff0000");
6
7     Rectangle rectangulo = new Rectangle(300, 300);
8     rectangulo.fillProperty().bind(Bindings.createObjectBinding(
9       () -> {
10         try {
11           return Color.web(color.getText());
12         } catch (IllegalArgumentException e) {
13           return Color.RED;
14         }
15       },
16       color.textProperty()
17     ));
18
19     stage.setScene(new Scene(new VBox(
20       new HBox(etiqueta, color),
21       rectangulo
22     )));
23     stage.show();
24   }
25 }
```


dependencia
--->
Agregación

⊙ como main nunca se instancia, no es correcto decir que es dueño de nada

1. a)

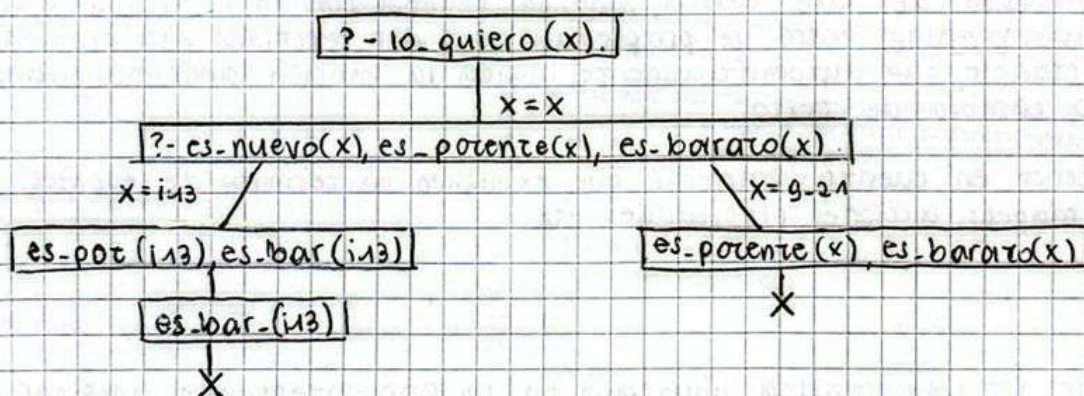


b) Hay polimorfismo presente en varias partes del código:
 • con la clase List (polimorfismo paramétrico/generic)

• con la clase Buffered Reader que recibe una "instancia" de la clase Reader, lo cual, al ser abstracta, en realidad implicará que reciba cualquier otra que extienda Reader (Ej: Buffered Reader, File Reader, InputStream Reader).

2. Se viola el principio OCP ya que, si se quiere agregar un nuevo formato, hay que modificar el código existente.
 Esto podría resolverse aprovechando los enums de Java (con comportamiento) o utilizando herencia.

3.



La consulta falla → Daí False

4.

determinística(x).

y devuelve Func.

• con-registro es puro y de orden superior (recibe una función F)

• La Fn [x] en la línea 4 es impura (invoca swap!)

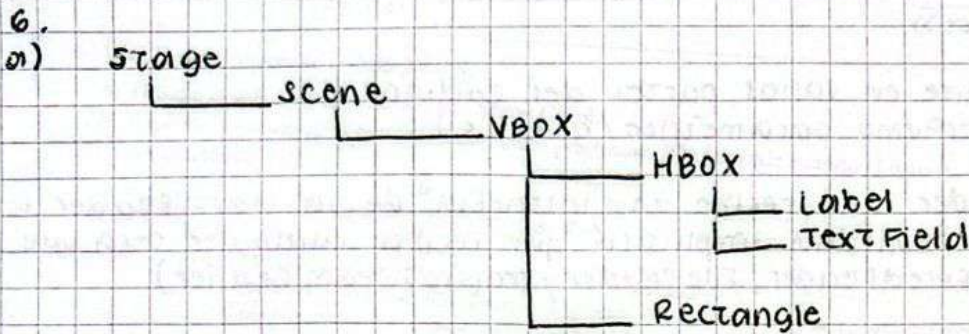
• La Fn [x] en la línea 9 es pura (sólo hace una multiplicación).

• La salida del programa no es determinística, ya que los Future se ejecutarán en orden aleatorio, por lo que el contenido de log puede variar entre ejecuciones.

⊙ Porque devuelve la función de la línea 4, no la ejecuta!

$$\begin{aligned}
 5. & (\lambda x. \lambda f. f x) (\lambda q. f) (\lambda a. a a) \\
 & =_{\alpha} (\lambda x. \lambda w. w x) (\lambda q. f) (\lambda a. a a) \\
 & =_{\beta} (\lambda w. w (\lambda q. f)) (\lambda a. a a) \\
 & =_{\beta} (\lambda a. a a) (\lambda q. f) \\
 & =_{\beta} (\lambda q. f) (\lambda q. f)
 \end{aligned}$$

$$=_{\beta} f$$



b) El programa recibe un color a través de un TextField en formato #RRGGBB.

Se crea un rectángulo que se rellenará con el color que se recibe por parámetro en caso de que exista, sino se rellena con rojo. Esto se logra mediante un binding entre la propiedad text del TextField y la propiedad fill de Rectangle, que automáticamente invoca la función anónima cuando el usuario cambia de texto.

cosas a tener en cuenta: palabras que expliquen en términos de eventos: bindings, mapeos, acciones, propiedades, etc.



→ Árbol de BT: la consulta buscará en la BDD intentando unificar la expresión con los hechos de los que tiene conocimiento para hallar el correcto utilizando backtracking.

(y explicar qué pasa con la consulta).